

1. Диаграмма вариантов использования ДП 01.00.ГЧ

2. Логическая схема базы данных ДП 02.00.ГЧ

3. Диаграмма развёртывания ДП 03.00.ГЧ

4. Блок-схема алгоритма оплаты абонемента ДП 04.00.ГЧ

5. Блок-схема алгоритма онлайн-тренировки ДП 05.00.ГЧ

6. Скриншот работы приложения ДП 06.00.ГЧ

Приложение А. Листинг скрипта создания базы данных

```
CREATE DATABASE "fitness_center_db";
\c "fitness_center_db";

-- Основные таблицы
CREATE TABLE abonement (
    id uuid PRIMARY KEY,
    title varchar(255),
    validity varchar(255),
    visiting_time varchar(255),
    photo varchar(255),
    price integer,
    created_time timestamptz,
    updated_time timestamptz,
    stripe_price_id varchar
);

CREATE TABLE service (
    id uuid PRIMARY KEY,
    title varchar(255),
    photo varchar(255),
    created_time timestamptz,
    updated_time timestamptz
);

CREATE TABLE coach (
    id uuid PRIMARY KEY,
    name varchar(255) NOT NULL,
    description varchar(255) NOT NULL,
    photo varchar(255),
    created_time timestamptz,
    updated_time timestamptz
);

CREATE TABLE review (
    id uuid PRIMARY KEY,
    body varchar(255) NOT NULL,
    created_time timestamptz,
    updated_time timestamptz,
    user_id uuid REFERENCES "user"(id) ON DELETE CASCADE
);

CREATE TABLE "user" (
    id uuid PRIMARY KEY,
    email varchar(255) NOT NULL,
    role varchar(255) NOT NULL,
    password_hash varchar(255) NOT NULL,
    photo varchar(255),
    name varchar(255) NOT NULL,
    created_time timestamptz,
    updated_time timestamptz
);
```

```

-- Вспомогательные связи
CREATE TABLE abonement_service (
    abonement_id uuid REFERENCES abonement(id) ON DELETE CASCADE,
    service_id uuid REFERENCES service(id) ON DELETE CASCADE,
    PRIMARY KEY (abonement_id, service_id)
);

CREATE TABLE coach_service (
    coach_id uuid REFERENCES coach(id) ON DELETE CASCADE,
    service_id uuid REFERENCES service(id) ON DELETE CASCADE,
    PRIMARY KEY (coach_id, service_id)
);

CREATE TABLE coach_review (
    coach_id uuid REFERENCES coach(id) ON DELETE CASCADE,
    review_id uuid REFERENCES review(id) ON DELETE CASCADE,
    PRIMARY KEY (coach_id, review_id)
);

-- Заказы и сессии
CREATE TABLE "order" (
    id uuid PRIMARY KEY,
    abonement_id uuid REFERENCES abonement(id) ON DELETE CASCADE,
    user_id uuid REFERENCES "user"(id) ON DELETE CASCADE,
    status varchar NOT NULL,
    created_time timestamptz,
    updated_time timestamptz,
    expiration_time timestamptz
);

CREATE TABLE refresh_sessions (
    id uuid PRIMARY KEY,
    user_id uuid REFERENCES "user"(id) ON DELETE CASCADE,
    refresh_token varchar(400) NOT NULL,
    finger_print varchar(32) NOT NULL,
    created_time timestamptz,
    updated_time timestamptz
);

CREATE TABLE schema_migrations (
    version bigint PRIMARY KEY,
    dirty boolean NOT NULL
); CREATE TABLE training (
    id uuid PRIMARY KEY,
    time_from timestamp NOT NULL,
    time_until timestamp NOT NULL,
    status training_status NOT NULL,
    coach_id uuid REFERENCES coach ON DELETE CASCADE,
    client_id uuid REFERENCES "user" ON DELETE
CASCADE, created_time timestamp NOT NULL, updated_time timestamp
NOT NULL
); GRANT ALL ON SCHEMA public TO fitness_center_admin;

```


Приложение Б. Листинг Protobuf SSO сервиса

```

syntax="proto3";
package fitness_center.sso;
option go_package="FitnessCenter.Protobuf.sso";
import "google/Protobuf/empty.proto";
import "user.proto";
service SSO {
    rpc SignUp(SignUpRequest) returns (SignUpResponse);
    rpc SignIn(SignInRequest) returns (SignInResponse);
    rpc Logout(LogoutRequest) returns (google.Protobuf.Empty);
    rpc Refresh(RefreshRequest) returns (RefreshResponse);}
message SignUpRequest {
    string name=1;
    string email=2;
    string password=3;
    string finger_print=4;}
message SignUpResponse {
    string access_token=1;
    string refresh_token=2;
    string access_token_expiration=3;
    string refresh_token_expiration=4;
    fitness_center.user.UserObject user=5;}
message SignInRequest {
    string email=1;
    string password=2;
    string finger_print=3;}
message SignInResponse {
    string access_token=1;
    string refresh_token=2;
    string access_token_expiration=3;
    string refresh_token_expiration=4;
    fitness_center.user.UserObject user=5;}
message LogoutRequest {
    string refresh_token=1;}
message RefreshRequest {
    string finger_print=1;
    string refresh_token=2;}
message RefreshResponse {
    string access_token=1;
    string refresh_token=2;
    string access_token_expiration=3;
    string refresh_token_expiration=4;
    fitness_center.user.UserObject user=5;
}

```

Приложение В. Листинг CreateCheckoutSession

```

func (o OrdergRPC) CreateCheckoutSession(ctx context.Context,
request *orderProtobuf.CreateCheckoutSessionRequest)
(*orderProtobuf.CreateCheckoutSessionResponse, error) {

    stripe.Key = o.StripeKey

    domain := o.clientDomain

    params := &stripe.CheckoutSessionParams{

        LineItems: []*stripe.CheckoutSessionLineItemParams{

            {Price: stripe.String(request.StripePriceId),
Quantity: stripe.Int64(1)},

        },

        Mode:
stripe.String(string(stripe.CheckoutSessionModePayment)),

        SuccessURL: stripe.String(domain),

        CancelURL: stripe.String(domain),

        Metadata: map[string]string{

            "client_id": request.ClientId,

            "abonement_id": request.AbonementId,

        },

    }

    s, err := session.New(params)

    if err != nil {

        log.Printf("session.New: %v", err)

    }

    return
&orderProtobuf.CreateCheckoutSessionResponse{SessionUrl: s.ID},
nil
}

```

Приложение Г. Листинг Webhook Stripe

```

func (h *Handler) HandleCheckoutSessionCompleted(c *gin.Context)
{
    payload, err := ioutil.ReadAll(c.Request.Body)

    if err != nil {      c.JSON(http.StatusInternalServerError,
gin.H{"error": "Failed to read body"})return}

    event, err := webhook.ConstructEvent(payload,
c.Request.Header.Get("Stripe-Signature"), endpointSecret)if err
!= nil {log.Printf("Webhook signature verification failed: %v",
err)}

    c.JSON(http.StatusBadRequest, gin.H{"error": "Invalid
signature"})      return}

    var clientId, abonementId string switch event.Type {

    case "checkout.session.completed":

        var session stripe.CheckoutSession

        if err := json.Unmarshal(event.Data.Raw, &session);
err != nil {

            log.Printf("Failed to parse session: %v", err)

            c.JSON(http.StatusBadRequest, gin.H{"error":
"Failed to parse event"})      return}

            clientId = session.Metadata["client_id"]abonementId =
session.Metadata["abonement_id"]

        default:

            log.Printf("Unhandled event type: %s", event.Type)}

        req := &orderGRPC.CreateOrderRequest{OrderDataForCreate:
&orderGRPC.OrderDataForCreate{UserId: clientId, AbonementId:
abonementId}}

        order, err := (*h.orderClient).CreateOrder(context.TODO(),
req)

        if err != nil {c.JSON(http.StatusInternalServerError,
gin.H{"error": err})return    }

        c.JSON(http.StatusOK, gin.H{"order": order.GetOrderObject()})}

```

Приложение Д. Листинг клиентской части онлайн-тренировок

```

const Room = (props) => {
  const { trainingId, coachId, coachName, clientName } =
useParams();

  let      currentUser      =      useSelector((state)      =>
state.userSliceMode.user);

  const navigate = useNavigate();

  //our camera and micro
const userVideo = useRef();
const userStream = useRef();

const partnerVideo = useRef();
const peerRef = useRef();

//ws obj
const websocketRef = useRef();

const [cameras, setCameras] = useState([]);
const [microphones, setMicrophones] = useState([]);
const [selectedCamera, setSelectedCamera] = useState("");
const      [selectedMicrophone,      setSelectedMicrophone]      =
useState("");
const      [isOpenMediaSelected,      setIsOpenMediaSelected]      =
useState("");

const [isCameraOn, setIsCameraOn] = useState(true);
const [isMicrophoneOn, setIsMicrophoneOn] = useState(true);

const toggleCamera = () => {
  if (userStream.current) {
    userStream.current.getVideoTracks().forEach((track) => {
      track.enabled = !isCameraOn;
    });
    setIsCameraOn(!isCameraOn);
  }
};

const toggleMicrophone = () => {
  if (userStream.current) {
    userStream.current.getAudioTracks().forEach((track) => {
      track.enabled = !isMicrophoneOn;
    });
    setIsMicrophoneOn(!isMicrophoneOn);
  }
};

const initDevices = async () => {
  try {
    // Запрашиваем разрешение на доступ к камере и микрофону

```

```

    await navigator.mediaDevices.getUserMedia({ video: true,
audio: true });

    // После получения доступа перечисляем устройства
    const allDevices = await
navigator.mediaDevices.enumerateDevices();

    setCameras(allDevices.filter((d) => d.kind ===
"videoinput"));
    setMicrophones(allDevices.filter((d) => d.kind ===
"audioinput"));
  } catch (error) {
    console.error("Ошибка доступа к устройствам:", error);
    ShowErrorMessage("Необходимо дать доступ к устройствам
ввода и вывода");
  }
};

const openMedia = async (videoDeviceId, audioDeviceId) => {
  const constraints = {
    video: videoDeviceId ? { deviceId: { exact: videoDeviceId
} } : true,
    audio: {
      deviceId: audioDeviceId ? { exact: audioDeviceId } :
undefined,
      echoCancellation: true, // Включение подавления эха
      noiseSuppression: true, // Подавление шума
    },
  };

  navigator.mediaDevices
    .getUserMedia(constraints)
    .then((stream) => {
      userVideo.current.srcObject = stream;
      userStream.current = stream;

      setIsOpenMediaSelected(true);

      const accessToken = inMemoryJWT.getToken();

      websocketRef.current = new WebSocket(
        config.TRAINING_JOIN_API_URL +
`/training/join/${trainingId}?coachId=${coachId}&token=${accessT
oken}`,
      );

      websocketRef.current.addEventListener("open", () => {
        websocketRef.current.send(JSON.stringify({ join:
true }));
      });

      websocketRef.current.addEventListener("message", async

```

```

(e) => {
    const message = JSON.parse(e.data);

    console.log("message: " + JSON.stringify(e));

    if (message.join) {
        callUser();
    }

    if (message.offer) {
        handleOffer(message.offer);
    }

    if (message.answer) {
        console.log("Receiving Answer");
        peerRef.current.setRemoteDescription(
            new RTCSessionDescription(message.answer),
        );
    }

    if (message.iceCandidate) {
        console.log("Receiving and Adding ICE Candidate");
        try {
            await
peerRef.current.addIceCandidate(message.iceCandidate);
        } catch (err) {
            console.log("Error Receiving ICE Candidate",
err);
        }
    }

    if (message.end) {
        if (userStream.current) {
            userStream.current.getTracks().forEach((track)
=> track.stop());
            userStream.current = null;
        }

        if (userVideo.current) userVideo.current.srcObject
= null;
        if (partnerVideo.current) (partnerVideo.current)
partnerVideo.current.srcObject = null;

        if (peerRef.current) {
            peerRef.current.close();
            peerRef.current = null;
        }

        if (websocketRef.current) {
            websocketRef.current.close();
            websocketRef.current = null;
        }
    }
}

```

```

        navigate("/main");
    }
    });
})
.catch((e) => {
    ShowErrorMessage("Невалидные устройства ввода или
ВЫВОДА");
});
};

useEffect(() => {
    initDevices().then((r) => console.log("devices
received"));

    return () => {
        console.log("Cleaning up resources...");

        // Остановка всех треков в медиа-потоке пользователя
        if (userStream.current) {
            userStream.current.getTracks().forEach((track) =>
track.stop());
            userStream.current = null;
        }

        // Очистка видеоэлементов
        if (userVideo.current) userVideo.current.srcObject =
null;
        if (partnerVideo.current) partnerVideo.current.srcObject
= null;

        // Закрытие WebSocket соединения
        if (websocketRef.current) {
            websocketRef.current.close();
            websocketRef.current = null;
        }

        // Закрытие и очистка PeerConnection
        if (peerRef.current) {
            peerRef.current.close();
            peerRef.current = null;
        }
    };
}, []);

const callUser = () => {
    console.log("Calling Other User");
    peerRef.current = createPeer();

    userStream.current.getTracks().forEach((track) => {
        peerRef.current.addTrack(track, userStream.current);
    });
};
};

```

```

const handleOffer = async (offer) => {
  console.log("Received Offer, Creating Answer");
  peerRef.current = createPeer();

  await peerRef.current.setRemoteDescription(
    new RTCSessionDescription(offer),
  );

  userStream.current.getTracks().forEach((track) => {
    peerRef.current.addTrack(track, userStream.current);
  });

  const answer = await peerRef.current.createAnswer();
  await peerRef.current.setLocalDescription(answer);

  websocketRef.current.send(
    JSON.stringify({
      peerRef.current.localDescription
    }),
  );
};

const createPeer = () => {
  console.log("Creating Peer Connection");
  const peer = new RTCPeerConnection({
    iceServers: [{ urls: "stun:stun.l.google.com:19302" }],
  });

  peer.onnegotiationneeded = handleNegotiationNeeded;
  peer.onicecandidate = handleIceCandidateEvent;
  peer.ontrack = handleTrackEvent;

  return peer;
};

const handleNegotiationNeeded = async () => {
  console.log("Creating Offer");

  try {
    const myOffer = await peerRef.current.createOffer();
    await peerRef.current.setLocalDescription(myOffer);

    websocketRef.current.send(
      JSON.stringify({
        peerRef.current.localDescription
      }),
    );
  } catch (err) {}
};

const handleIceCandidateEvent = (e) => {
  console.log("Found Ice Candidate");
  if (e.candidate) {
    console.log(e.candidate);
    websocketRef.current.send(JSON.stringify({ iceCandidate:

```



```

e.candidate }));
    }
  };

  const handleTrackEvent = (e) => {
    console.log("Received Tracks");
    partnerVideo.current.srcObject = e.streams[0];
  };

  const videoStyle = {
    width: "500px",
    height: "300px",
    borderWidth: "4px",
    borderColor: "rgba(160, 147, 197, 1)",
    borderStyle: "solid",
    borderRadius: "10px",
    objectFit: "cover",
  };

  return currentUser.role === "client" || currentUser.role ===
"coach" ? (
    <div
      style={{
        display: "flex",
        justifyContent: "center",
        alignItems: "center",
        background: "rgba(117,100,163,255)",
        flexDirection: "column",
        width: "100%",
      }}
    >
      <div
        style={{
          display: "flex",
          alignItems: "center",
          justifyContent: "center",
          gap: "40px",
          background: "rgba(117,100,163,255)",
          width: "100%",
        }}
      >
        <div style={{ display: "flex", flexDirection: "column"
}}>
          <video
            style={{ ...videoStyle, transform: "scaleX(-1)" }}
            autoPlay
            ref={userVideo}
          ></video>
          <div
            style={{
              paddingTop: "10px",
              display: "flex",
              justifyContent: "center",

```

```

        fontSize: "24px",
      }}
    >
      {currentUser.role === "client" ? clientName :
coachName}
    </div>
  </div>

  <div style={{ display: "flex", flexDirection: "column"
}}>
    <video
      style={{ ...videoStyle, transform: "scaleX(-1)" }}
      autoPlay
      ref={partnerVideo}
    ></video>
    <div
      style={{
        paddingTop: "10px",
        display: "flex",
        justifyContent: "center",
        fontSize: "24px",
      }}
    >
      {currentUser.role === "client" ? coachName :
clientName}
    </div>
  </div>
  </div>
  {!isOpenMediaSelected && (
    <div
      style={{
        marginTop: "10px",
        display: "flex",
        width: "89%",
        gap: "15px",
      }}
    >
      <select
        style={{
          width: "150px",
          height: "40px",
          color: "white",
          background: "rgba(160, 147, 197, 1)",
          border: "none",
          borderRadius: "5px",
          padding: "5px 10px",
          fontSize: "16px",
          cursor: "pointer",
          outline: "none",
        }}
        onChange={ (e)
setSelectedCamera (e.target.value) }
      >

```

=>

```

        {cameras.map((camera) => (
            <option                                key={camera.deviceId}
value={camera.deviceId}>
                {camera.label || `Камера ${camera.deviceId}`}
            </option>
        ))}
    </select>
    <select
        style={{
            width: "150px",
            height: "40px",
            color: "white",
            background: "rgba(160, 147, 197, 1)",
            border: "none",
            borderRadius: "5px",
            padding: "5px 10px",
            fontSize: "16px",
            cursor: "pointer",
            outline: "none",
        }}
        onChange={ (e)                                ==>
setSelectedMicrophone(e.target.value) }
    >
        {microphones.map((mic) => (
            <option key={mic.deviceId} value={mic.deviceId}>
                {mic.label || `Микрофон ${mic.deviceId}`}
            </option>
        ))}
    </select>
    <Button
        style={{
            width: "160px",
            height: "40px",
            color: "white",
            background: "rgba(160, 147, 197, 1)",
            border: "none",
            borderRadius: "5px",
            padding: "5px 10px",
            fontSize: "16px",
            cursor: "pointer",
            outline: "none",
        }}
        onClick={ ()                                ==>          openMedia(selectedCamera,
selectedMicrophone) }
    >
        Начать звонок
    </Button>
</div>
)}
{isOpenMediaSelected && (
    <div
        style={{
            marginTop: "10px",

```

```

        display: "flex",
        width: "89%",
        gap: "15px",
      }}
    >
      <Button
        onClick={toggleCamera}
        style={{ background: "rgba(160, 147, 197, 1)",
color: "white" }}
      >
        {isCameraOn ? "Выключить камеру" : "Включить
камеру"}
      </Button>
      <Button
        onClick={toggleMicrophone}
        style={{
          background: "rgba(160, 147, 197, 1)",
          color: "white",
          fontSize: "16px",
        }}
      >
        {isMicrophoneOn ? "Выключить микрофон" : "Включить
микрофон"}
      </Button>
    </div>
  )}
</div>
) : null;
};

```